School of Computer Science: Assessment Brief

Module title	Web Services and Web Data
Module code	COMP/XJCO3011
Assignment title	Coursework 2: Search Engine Tool
Assignment type and description	Coursework – individual practical project with video demonstration
Rationale	This coursework will help develop your skills in building web crawlers, indexers, and web search algorithms. It assesses your ability to understand how search engines work, write efficient code for web crawling, indexing, and query processing, learn various techniques of storing words in search indices, and implement algorithms for ranking and retrieving search terms from indices.
Word limit and guidance	5-minute video demonstration is the primary assessment component, accompanied by a GitHub repository and index file submission.
Use of GenAI in this assessment	**GREEN: AI has an integral role and should be used as part of the assessment.** You can use GenAI as a primary tool throughout the assessment process. A core part of this assessment (15% of the grade) is your critical reflection on GenAI usage in your video demonstration. You must discuss specific examples of where GenAI helped or hindered your work, analyse the quality of AI-generated code, and reflect on how using (or not using) GenAI affected your learning. Non-declared or misleading declaration of GenAI use constitutes academic misconduct.
Weighting	30% of overall module mark

Submission deadline	12 May 2026
Submission method	Electronic submission via Minerva: video demonstration link, GitHub repository URL, and index file
Feedback provision	Electronic via Minerva
Learning outcomes assessed	• Understanding of search engine mechanisms • Efficient coding for web crawling, indexing, and query processing • Knowledge of search index storage techniques • Implementation of ranking and retrieval algorithms • Critical evaluation of Generative AI tools • Comprehensive testing and version control practices
Module lead	Dr Ammar Alsalka (M.A.Alsalka@leeds.ac.uk)
Other Staff contact	Mr Omar Choudhry (O.Choudhry@leeds.ac.uk) Dr Guilin Zhao (guilinzhao@swjtu.edu.cn)

1. **Assignment guidance**

In this assignment, you will develop a search tool that can:

(a) Crawl the pages of a website;

(b) Create an inverted index of all word occurrences in the pages of the website;

(c) Allow the user to find pages containing certain search terms.

Your work will be assessed through a **5-minute video demonstration** that showcases your implementation, explains your design decisions, demonstrates testing, and provides a critical evaluation of any GenAI tools used.

(a) Target Website

The website you will use for this project is:

https://quotes.toscrape.com/

This website contains a collection of common quotes and is purpose-built to help people learn web scraping.

(b) Important Requirements

- **Politeness Window**: You must observe a politeness window of at least **6 seconds** between successive requests to the website.
- **Inverted Index**: An inverted index that stores statistics (e.g. frequency, position, etc) of each word in each page must be created by the tool as it crawls the pages of the website.
- **Case Sensitivity**: For simplicity, assume that the search is not case sensitive, so 'Good' is the same word as 'good'.

(c) Technical Requirements

You should use **Python** to implement the search tool. It is recommended that you use:

- **Requests library** (http://docs.python-requests.org/en/master/) for composing HTTP requests;

- **Beautiful Soup library** (https://www.crummy.com/software/BeautifulSoup/bs4/doc/) to parse HTML pages.

(d) Command-Line Interface

The search tool must have a command-line interface (shell) and provide the following commands:

(i) build – This command instructs the search tool to crawl the website, build the index, and save the resulting index into the file system. For simplicity, you can save the entire index to a single file.

```
> build
```

(ii) load – This command loads the index from the file system. Obviously, this command will only work if the index has been created previously with the 'build' command.

```
> load
```

(iii) print – This command prints the inverted index for a particular word. The following example will print the inverted index for the word 'nonsense'.

```
> print  nonsense
```

(iv) find – This command is used to find a given query phrase in the inverted index and returns a list of all pages that contain it. The first example will return a list of all pages containing the word 'indifference'. The second example will return all pages containing the words 'good' and 'friends'.

```
> find  indifference

> find good friends
```

2. **Use of GenAI**

   This assessment is **Green Category**. **AI tools may be used with proper declaration and critical evaluation.** You are permitted to use GenAI under the following conditions:

   - **Declaration**: You must clearly state which AI tools you used and for what purposes in your video demonstration.
   - **Critical Evaluation (15% of grade)**: A core part of this assessment is your critical reflection on GenAI usage. In your video, you must:
     - Discuss specific examples of where GenAI helped or hindered your work
     - Analyse the quality and correctness of AI-generated code/ suggestions
     - Reflect on how using (or not using) GenAI affected your learning
     - Discuss any challenges in understanding or debugging AI-generated code
     - Evaluate the impact on your development process and time management
   - **Understanding**: You must demonstrate complete understanding of all code in your submission, whether AI-assisted or not. You should be able to explain every line of code and justify all design decisions.
   - **If NOT using GenAI**: If you chose not to use GenAI tools, discuss this decision in your video and reflect on your development experience without AI assistance.

   Example GenAI Evaluation Topics:

   Good critical evaluations might discuss:

   - ”AI suggested an incorrect data structure for my inverted index, which I debugged and improved”
   - ”AI helped me understand BeautifulSoup’s API, but I had to modify its example code significantly”

- ”I chose not to use AI for core algorithms to ensure I truly understood the implementation”
- ”AI-generated tests missed several edge cases I had to identify and add manually”

Important: Non-declaration or misleading declaration of GenAI use constitutes academic misconduct.

To ensure privacy and data security, please use only the University’s **secure Copilot access** when engaging with AI tools.

3. **Assessment tasks**

   Write code and create a 5-minute video demonstration presenting your design and implementation.

   (i) Video Demonstration Requirements

   Your **5-minute video demonstration** is the primary assessment component. The video must be uploaded to an accessible platform (Google Drive, YouTube, OneDrive, etc.) and the link submitted via Minerva. **Please check that your link works in the submission, by opening it in an incognito browser window.**

   What to Include in Your Video:

   Your video demonstration must cover the following components (please see recommended times allocated for each section in brackets):

   (a) **Live Demonstration** (2 minutes)
   - Show your search tool running all four commands: build, load, print, and find
   - Demonstrate handling of multi-word queries
   - Show the tool handling edge cases (e.g., non-existent words, empty queries)

   (b) **Code Walkthrough & Design Decisions** (1.5 minutes)
   - Explain your choice of data structures for the inverted index
   - Show key sections of your code (crawler, indexer, search logic)
   - **Justify your implementation choices and any trade-offs made**

   (c) **Testing Demonstration** (0.5 minutes)
   - Show your test suite running
   - Explain your testing strategy and coverage

   (d) **Version Control** (0.5 minutes)
   - Show your Git commit history demonstrating regular, incremental development
   - Briefly explain your development workflow

   (e) **GenAI Critical Evaluation** (0.5 minutes)

- Critically evaluate any GenAI tools used (if applicable)
- Discuss specific examples of where GenAI helped or hindered your work
- Reflect on your learning experience with/without GenAI

Video Technical Requirements:

- **Duration**: Maximum 5 minutes (videos exceeding 5 minutes will have marks deducted)
- **Format**: MP4, MOV, or similar standard video format
- **Resolution**: 720p minimum
- **Hosting**: Upload to Google Drive, YouTube (make it unlisted, **not private**), OneDrive (shared with "Anyone at the University of Leeds"), or similar platform
- **Accessibility**: Ensure the video link is accessible to markers (check sharing permissions)
- **Audio & Visual Quality**: Clear audio narration and legible screen recording
- **Screen Recording**: Use appropriate screen recording software to show your code and terminal

(ii) GitHub Repository Structure

Your GitHub repository should be organised as follows:

```
repository-name/
  src/
      crawler.py
      indexer.py
      search.py
      main.py
  tests/
      test_crawler.py
      test_indexer.py
      test_search.py
  data/
```

[compiled index file]
requirements.txt
README.md

Your README.md must include:

- Project overview and purpose
- Installation/setup instructions
- Usage examples for all four commands
- Testing instructions
- Any dependencies and how to install them

4. **General guidance and study support**

Detailed information and guidance on building search tools are available in the module's learning resources on Minerva.

(a) Recommended Resources

- Module lecture slides on web crawling, indexing, and search algorithms;
- Python Requests library documentation: http://docs.python-requests.org/en/master/;
- Beautiful Soup library documentation: https://www.crummy.com/software/BeautifulSoup/bs4/doc/;
- Target website for practice: https://quotes.toscrape.com/.

(b) Getting Help

If you encounter difficulties or have questions about the assignment:

- Attend module practical lab sessions where teaching staff can provide guidance;
- Post questions on the module discussion forum on Minerva;
- Contact the module leader or teaching staff during office hours.

(c) Key Implementation Tips

- **Test incrementally**: Build and test each component (crawler, indexer, search) separately before integration;
- **Write tests as you go**: Don't leave testing until the end – build your test suite alongside your implementation;
- **Commit regularly**: Make frequent Git commits with meaningful messages to demonstrate your development process;
- **Handle errors gracefully**: Network requests can fail; implement appropriate error handling;
- **Document your code**: Clear comments and docstrings will help when creating your video explanation;
- **Respect the politeness window**: Always wait at least 6 seconds between requests;

- **Choose appropriate data structures**: Consider efficiency for both indexing and searching operations;
- **Plan your video**: Script or outline your video before recording to ensure you cover all required components within 5 minutes;
- **Practice your demonstration**: Do a dry run to check timing and ensure everything works smoothly.

(d) Video Recording Tips

- Use screen recording software like OBS Studio (free) or QuickTime (Mac)
- Prepare recorded videos in a PowerPoint presentation instead (you can also record the video using Microsoft PowerPoint itself).
- Test your audio levels before recording the full video
- Use a quiet environment with minimal background noise
- Consider using a script or bullet points to stay on track
- If you make a mistake, you can edit the video or re-record
- Zoom in on code/terminal output so text is clearly readable
- Consider adding captions or text overlays for key points

5. Assessment criteria and marking process

Assessment Criteria	Weighting
Crawling Implementation Successfully crawls all pages, respects politeness window, handles errors	10%
Indexing Implementation Creates correct inverted index with word statistics	10%
Storage & Retrieval (build/load commands) Saves and loads index correctly from file system	8%
Search Functionality (print/find commands) Correctly implements print and find commands for single and multi-word queries	12%
Testing & Test Coverage Comprehensive test suite covering edge cases, documented testing strategy	20%
Code Quality & Documentation Clear code structure, appropriate data structures, inline comments, README	10%
Version Control & Git Practices Regular commits with meaningful messages, demonstrates incremental development	5%
Video Demonstration Quality Clear presentation, well-structured, covers all required components, good timing	10%
GenAI Critical Evaluation Thoughtful reflection, specific examples, learning insights	15%
Total	**100%**

The following grading bands describe what is expected at each performance level. Use these as a guide when preparing your video demonstration.

Grade Band	Expectations
40–49 (Pass)	• Basic working implementation of all four commands • Crawls most pages but may miss some or have minor errors • Inverted index created with basic structure • Some tests present but limited coverage (¡50%) • Git history shows development, but commits may be irregular or poorly described • Video demonstrates basic functionality but lacks a clear explanation • GenAI evaluation present but superficial (e.g., ”I used ChatGPT and it was helpful”) • Code has minimal documentation, some structural issues
50–59 (Satisfactory)	• All four commands work correctly on standard inputs • Crawler successfully retrieves all pages, respects politeness window • Inverted index correctly stores word statistics • Test suite with reasonable coverage (50–70%), covers main functionality • Regular Git commits with descriptive messages • Video clearly demonstrates all functionality, adequate explanations • GenAI evaluation discusses specific uses (e.g., ”Used for debugging X issue”) • Code is readable with some documentation, basic README present • Some basic error handling implemented

Continued on next page

Grade Band	Expectations
60–69 (Good)	• Robust implementation handling edge cases (empty queries, special characters, etc.) • Efficient crawler with proper error handling for network issues • Well-designed inverted index with appropriate data structures • Comprehensive test suite (70–85% coverage) including edge cases • Consistent Git workflow with meaningful commit messages, shows iterative development • Video is well-structured, clearly explains design decisions and trade-offs • GenAI evaluation is thoughtful, discusses both benefits and limitations with examples • Clean, well-documented code with clear structure and inline comments • Good README with setup instructions and usage examples

Continued on next page

Grade Band	Expectations
70–79 (Very Good)	• Excellent implementation with optimised data structures and algorithms • Crawler efficiently handles various HTML structures and error conditions • Sophisticated inverted index design with justification for choices • Extensive test suite (¿85% coverage) with unit, integration, and performance tests • Exemplary Git practices: feature branches, clear commit history, logical progression • Professional video presentation with clear narrative, good pacing, and visual aids • Insightful GenAI evaluation critically analysing the impact on the learning and development process • High-quality code following Python best practices (PEP 8), modular design • Comprehensive README with architecture overview and design rationale • Code includes defensive programming and graceful error recovery

Continued on next page

Grade Band	Expectations
80–100 (Excellent to Outstanding)	• Exceptional implementation demonstrating deep understanding and innovation • Advanced features beyond requirements (e.g., TF-IDF ranking, advanced query processing, query suggestions) • Highly optimised algorithms with complexity analysis and benchmarking • Professional-grade test suite with extensive coverage, mocking, and automated testing pipeline • Professional Git workflow with semantic commits, tags/releases, and branching strategy • Outstanding video: engaging presentation, demonstrates mastery, discusses algorithmic trade-offs • Sophisticated GenAI evaluation showing critical thinking about AI's role in software development, discusses ethical considerations or learning implications • Publication-quality code: exemplary structure, comprehensive documentation, type hints, docstrings • Professional README comparable to open-source projects • Evidence of research into search engine algorithms and modern practices • (90–100): Novel contributions or particularly creative solutions to challenges

6. **Presentation and referencing**

   The quality of communication in your video and code documentation will be assessed. As a minimum, you must ensure:

   - Your explanation follows a logical structure;
   - You reference any external resources, libraries, or tutorials used;
   - Your narration clearly communicates your implementation approach;
   - Technical terminology is used appropriately.

7. **Submission requirements**

(i) What to Submit via Minerva

You must submit a **single text document (PDF or TXT)** via Minerva containing:

(a) **Video demonstration link**: URL to your 5-minute video (Google Drive, YouTube, OneDrive, etc.)

- Ensure sharing permissions are set correctly (accessible to anyone with the link)
- Test the link in an incognito/private browser window before submission

(b) **GitHub repository URL**: Link to your public GitHub repository containing:

- All source code files
- Comprehensive README.md with setup and usage instructions
- Test files and testing documentation
- Any auxiliary files needed to run the tool

(c) **Index file**: Attach the compiled index file generated by your search tool

- This can be uploaded as a separate attachment on Minerva
- Or include a download link if the file is too large

(ii) Late Submissions

Late submissions without approved mitigation will be penalised according to the University of Leeds guidelines:

- Late submissions without approved mitigation incur a **5% penalty per day**.
- **Maximum extension period for mitigating circumstances is 2 weeks.**

8. **Academic misconduct and plagiarism**

Leeds students are part of an academic community that shares ideas and develops new ones.

You need to learn how to work with others, interpret and present other people's ideas, and produce your own independent academic work. It is essential that you can distinguish between other people's work and your own, and correctly acknowledge others' work.

All students new to the University are expected to complete an online Academic Integrity tutorial and test, and all Leeds students should ensure that they are aware of the principles of Academic Integrity.

When you submit work for assessment, it is expected that it will meet the University's academic integrity standards.

If you do not understand what these standards are, or how they apply to your work, then please ask the module teaching staff for further guidance.

By submitting this assignment, you are confirming that:

- The work is a true expression of your own work and understanding;
- You have declared all GenAI tools used (or stated that none were used);
- You can explain and justify all code and design decisions in your submission;
- You have given credit to others (including AI tools) where their work has contributed to yours.